

HHBKTendo – Spielesammlung

Eröffnung

Guten Tag, wir präsentieren heute **HHBKTendo** – eine Spielesammlung, die im Rahmen von Lernfeld 5 an der HHBK Düsseldorf entwickelt wurde.

Das Projekt besteht aus zwei Strategiespielen – **Bauernschach** und **Tic-Tac-Toe (4 gewinnt)** – jeweils auf einem 6×6-Spielfeld, gegen eine KI die auf dem MiniMax-Algorithmus basiert.

Technisch setzt das Projekt auf **Python 3.10+**, eine **SQLite-Datenbank** und eine **tkinter-Oberfläche** – alles ohne externe Abhängigkeiten außer Pillow für Animationen.

■ **Kurze Vorstellung des Teams vor dem Start, dann direkt in den Überblick.**

Projektüberblick

Das Projekt hat vier Kernbereiche:

- **Spielesammlung:** Zwei vollständige Strategiespiele auf einem 6×6-Feld. Beide Spiele sind komplett spielbar gegen eine KI.
 - **KI-Gegner:** Ein einziger, wiederverwendbarer MiniMax-Algorithmus mit Alpha-Beta-Pruning funktioniert für beide Spiele – kein duplizierter Code.
 - **Benutzerverwaltung:** Spieler können sich registrieren und einloggen. Passwörter werden mit SHA-256 und Salt gesichert. Alternativ ist auch ein Gast-Modus ohne Konto möglich.
 - **Bestenliste:** Alle Spielergebnisse werden in einer SQLite-Datenbank gespeichert und können nach Spiel und Schwierigkeitsgrad sortiert abgerufen werden.
- **Überblick-Folie – ruhig etwas langsamer sprechen, damit alle folgen können.**

Projektstruktur

Modularer Aufbau

Das Projekt ist in sieben Python-Dateien aufgeteilt, jede mit einer klar definierten Aufgabe:

- **main.py** – Der GUI-Einstiegspunkt. Verwaltet alle Screens und den globalen Zustand über das `app_state`-Dictionary.
 - **minimax.py** – Die KI-Engine. Generischer MiniMax mit Alpha-Beta-Pruning, nutzbar für jedes Spiel per Callbacks.
 - **pawn_chess.py / tictactoe.py** – Die eigentliche Spiellogik: erlaubte Züge berechnen, Züge anwenden, Gewinner ermitteln, Stellungen bewerten.
 - **auth.py** – Registrierung und Login mit SHA-256-Hashing und Salt.
 - **database.py** – Alle Datenbankoperationen: Nutzer anlegen, Ergebnisse speichern, Bestenliste abfragen.
 - **test_all.py** – 15 Unittests (TC-01 bis TC-15), pytest-kompatibel.
- **Auf die klare Trennung der Zuständigkeiten hinweisen – jede Datei hat genau eine Aufgabe.**

Benutzerverwaltung & Datenbank

auth.py · database.py

auth.py – Funktionen

- **register()**: Prüft ob der Username schon vergeben ist, hasht das Passwort mit SHA-256 + Salt und speichert den neuen Nutzer in der DB.
- **login()**: Lädt den gespeicherten Hash aus der DB, wendet denselben Salt an und vergleicht das Ergebnis.
- **save_session() / load_session()**: Schreiben bzw. lesen die user_id in eine session.json – so bleibt man beim nächsten Start eingeloggt.
- **logout()**: Löscht die session.json und setzt den app_state zurück.
- **is_guest()**: Gibt True zurück wenn kein Nutzer eingeloggt ist – Gäste sehen keine Statistiken.

SQLite-Tabellen

- **users**: id, username, password_hash, language, created_at
 - **results**: id, user_id (Fremdschlüssel), game, difficulty, won, played_at
- Kurz erwähnen dass die DB automatisch erstellt wird wenn sie noch nicht existiert.

Datenbank-Techniken

row_factory · Migration · SQL-Injection-Schutz

Wir nutzen drei wichtige Techniken für eine robuste Datenbankanbindung:

sqlite3.Row als row_factory

Statt Spalten per Index anzusprechen (`row[1]`) kann man sie per Name abrufen: `row['username']`. Das macht den Code lesbarer und weniger fehleranfällig wenn sich die Spaltenreihenfolge ändert. Mit `dict(row)` lässt sich das Ergebnis einfach in ein Dictionary umwandeln.

SQL-Injection-Schutz

Alle SQL-Abfragen verwenden **?-Platzhalter** statt String-Konkatenation. SQLite ersetzt die Platzhalter sicher und verhindert so, dass Nutzereingaben als SQL-Code interpretiert werden.

Schema-Migration

Wenn eine neue Spalte zur DB hinzugefügt wird, wird der ALTER TABLE-Befehl in einem try/except-Block ausgeführt. Falls die Spalte schon existiert, wird der Fehler ignoriert – so bleibt die DB abwärtskompatibel.

■ **Gutes Beispiel für defensives Programmieren – kurz den Sicherheitsaspekt betonen.**

Sichere Passwortspeicherung

SHA-256 + Salt

Passwörter werden niemals im Klartext gespeichert. Stattdessen wird ein **kryptografischer Hash** mit einem zufälligen Salt erzeugt:

- **os.urandom(16)** erzeugt 16 kryptografisch zufällige Bytes als Salt. Das ist wichtig, weil ein zufälliger Salt dafür sorgt, dass zwei gleiche Passwörter unterschiedliche Hashes erzeugen.
- **Rainbow-Tables** sind vorberechnete Tabellen mit Passwort-Hash-Paaren. Durch den einzigartigen Salt wird diese Angriffsmethode wirkungslos.
- **SHA-256** ist eine Einweg-Hashfunktion – aus dem Hash lässt sich das ursprüngliche Passwort nicht zurückberechnen.
- In der Datenbank wird nur **'salt:hash'** gespeichert – niemals das Passwort selbst.

Beim Login wird der gespeicherte Salt verwendet, das eingegebene Passwort erneut gehasht und das Ergebnis verglichen.

■ **Falls jemand fragt: SHA-256 ist für dieses Schulprojekt ausreichend. Produktive Systeme würden bcrypt oder argon2 verwenden.**

MiniMax-Algorithmus

mit Alpha-Beta-Pruning

Der MiniMax-Algorithmus ist die KI-Grundlage beider Spiele. Er funktioniert wie folgt:

- Der Algorithmus baut einen **Spielbaum** auf – er simuliert alle möglichen Züge bis zu einer festgelegten Tiefe und bewertet jeden Endzustand.
- Es gibt zwei Spieler: den **Maximizer** (die KI, die einen möglichst hohen Score will) und den **Minimizer** (der Mensch, der den Score möglichst niedrig halten will). Beide spielen optimal.
- **Alpha-Beta-Pruning** ist eine Optimierung: Äste des Spielbaums, die das Ergebnis nicht mehr beeinflussen können, werden abgeschnitten. Das spart Rechenzeit deutlich.

Schwierigkeitsgrade = Suchtiefe

- Tiefe 1 (Leicht) – kaum strategisch | Tiefe 3 (Schwer) – Standard
- Tiefe 5 (Meister) – sehr stark, kann mehrere Sekunden dauern
- **Visualisierung: Spielbaum mit Max/Min-Ebenen kurz erklären, dann auf Alpha-Beta eingehen.**

Callback-basiertes MiniMax-Design

Eine KI-Engine für alle Spiele

Das Herzstück des Designs: Die MiniMax-Funktion `get_best_move()` kennt kein spezifisches Spiel. Stattdessen bekommt sie drei Callback-Funktionen übergeben:

- **get_moves_fn** – gibt die Liste aller gültigen Züge für den aktuellen Spielzustand zurück.
- **apply_move_fn** – wendet einen Zug auf das Spielfeld an und gibt den neuen Zustand zurück.
- **evaluate_fn** – bewertet einen Spielzustand mit einem numerischen Score.

Diese drei Funktionen werden für jedes Spiel separat implementiert. Der MiniMax-Code selbst muss dafür nicht verändert werden.

Vorteile

- Kein duplizierter Code – dieselbe KI-Logik für beide Spiele.
 - Single Responsibility – KI und Spiellogik vollständig getrennt.
 - Einfach erweiterbar – ein drittes Spiel braucht nur neue Callbacks.
- Gutes Beispiel für Clean Code und das Open/Closed Principle.

Bauernschach

6×6 · Weiß (Mensch) vs. Schwarz (KI)

Bauernschach ist eine vereinfachte Schachvariante, bei der nur Bauern auf einem 6×6-Feld gespielt werden.

Aufbau

- Weiß (Mensch) startet in Reihe 6, Schwarz (KI) in Reihe 1. Beide Seiten haben je sechs Bauern.

Zugregeln

- Ein Bauer kann **ein Feld vorwärts** ziehen, wenn das Zielfeld frei ist.
- **Diagonal schlagen**: Ein Bauer kann diagonal nach vorne ziehen, wenn dort ein gegnerischer Bauer steht.
- Kein Doppelschritt – anders als im echten Schach.

Gewinnbedingungen

- Die gegnerische **Grundlinie erreichen** (Umwandlung).
- **Alle Figuren** des Gegners schlagen.
- Der Gegner kann **keinen Zug mehr** machen.

Bewertungsfunktion

- Figurenzahl + Vorwärtsfortschritt + Zentrumskontrolle.
- **Kurz zeigen wie das Spiel aussieht (Screenshot auf der Folie).**

Bewertungsfunktion Bauernschach

Heuristik: Figurenzahl · Fortschritt · Zentrum

Die Bewertungsfunktion gibt der KI einen numerischen Score für jede Spielposition. Ein **positiver Wert** bedeutet, die KI steht gut; ein negativer, dass der Mensch im Vorteil ist.

■ **Figurenzahl (+10 pro eigener Bauer):** Jeder eigene Bauer zählt +10, jeder gegnerische -10. Wer mehr Figuren hat, steht besser.

■ **Fortschritt (+row×2):** Je weiter ein Bauer der KI nach vorne gerückt ist, desto mehr Punkte bekommt er. Das motiviert die KI, vorwärtszuspielen.

■ **Zentrumskontrolle (+0 bis +3):** Felder in der Mitte des Bretts sind strategisch wertvoller. Die CENTER_BONUS-Matrix vergibt Boni von 0 (Rand) bis 3 (Zentrum).

■ **Kurz erklären warum eine gute Heuristik wichtiger ist als eine höhere Suchtiefe.**

Tic-Tac-Toe (4 gewinnt)

6×6 · Mensch (X) vs. KI (O)

Die zweite Spielvariante: Vier in einer Reihe auf einem 6×6-Feld – eine Erweiterung des klassischen Tic-Tac-Toe.

Spielregeln

- Mensch (X) und KI (O) setzen abwechselnd Steine, der Mensch beginnt.
- Wer zuerst **vier Steine in einer Reihe** hat – horizontal, vertikal oder diagonal – gewinnt.
- Bei vollem Feld ohne Gewinner: **Unentschieden**.

KI-Optimierungen

- **Kandidatenzüge:** Statt alle 36 leeren Felder zu prüfen, betrachtet die KI nur Felder, die an bereits besetzte Felder angrenzen. Das reduziert die Züge auf 8–15.
- **Center-Sorting:** Kandidaten werden nach Zentrumsnähe sortiert. Alpha-Beta findet dadurch früher gute Äste und schneidet mehr ab.

Bewertungsfunktion

- Dreier-Reihe: +50 / -50 · Zweier-Reihe: +10 / -10 · Mittelfeld-Bonus
- **Unentschieden kommt selten vor – erwähnen falls jemand fragt.**

TTT Optimierungen im Detail

Kandidatenzüge · Center-Sorting · shallow copy

Kandidatenzüge

In einem 6×6-Feld gibt es anfangs 36 mögliche Züge. Die KI schränkt das auf Felder ein, die direkt neben einem bereits besetzten Feld liegen. Das ist sinnvoll, weil ein Stein in der Mitte des Feldes ohne Nachbarn strategisch fast wertlos ist. Dadurch sinkt die Anzahl der zu prüfenden Züge auf typischerweise 8–15.

Center-Sorting

Kandidaten werden vor der Auswertung nach ihrer Entfernung zum Mittelpunkt sortiert. Zentrumsnähe ist ein gutes Qualitätsmerkmal für Züge. Da Alpha-Beta bessere Züge zuerst sehen und mehr Äste abschneiden kann, beschleunigt das die Suche erheblich.

shallow copy statt deepcopy

Das Spielfeld wird als Liste von Listen kopiert: `[row[:] for row in board]`. Das ist etwa 5× schneller als `deepcopy(board)`, weil nur die Zeilenlisten flach kopiert werden – ausreichend für ein 6×6-Feld mit ganzen Zahlen.

■ **Wer fragt: `deepcopy` wäre korrekt für verschachtelte Objekte, hier aber nicht nötig.**

GUI & Threading

daemon Thread · after(0, ...) · app_state

app_state – globaler Zustand

Statt Klassen wird der gesamte Programmzustand in einem einzigen Dictionary `app_state` gespeichert. Das enthält das aktuelle Spielfeld, den eingeloggtten Nutzer, die gewählte Schwierigkeit, und ob die KI gerade denkt.

KI-Zug in separatem Thread

Der MiniMax-Algorithmus kann bei höheren Schwierigkeitsgraden mehrere Sekunden rechnen. Würde er im Hauptthread laufen, würde das GUI einfrieren. Deshalb läuft der KI-Zug in einem separaten Thread.

- **daemon=True:** Der Thread wird automatisch beendet wenn das Fenster geschlossen wird – kein Aufräumen nötig.

after(0, callback) – Thread-Safety

tkinter ist nicht thread-safe. Widgets dürfen nur aus dem Hauptthread verändert werden. Deshalb ruft der KI-Thread am Ende `after(0, fn)` auf – das marshallt den GUI-Update sicher in den tkinter-Hauptthread zurück.

- **Gute technische Frage vorhersagen: 'Warum kann man tkinter nicht aus dem Thread aufrufen?' → Event-Loop-Architektur.**

Unittests

TC-01 bis TC-15 · test_all.py · pytest-kompatibel

Das Projekt umfasst 15 Unittests, die alle Kernfunktionen abdecken:

- **TC-01–03 (Passwort-Hashing):** Prüft ob ein Hash korrekt generiert wird, ob die Verifikation funktioniert, und ob zwei gleiche Passwörter verschiedene Hashes erzeugen.
 - **TC-04–06 (Benutzer-DB):** Registrierung eines neuen Nutzers, Erkennung von Duplikaten, Abfrage des Nutzerprofils.
 - **TC-07–08 (Spielergebnisse):** Speichern eines Ergebnisses, korrekte Bestenliste abrufen.
 - **TC-09–10 (Bauernschach):** Gültige Züge generieren, Zug anwenden, Gewinner erkennen.
 - **TC-11–12 (Tic-Tac-Toe):** Vier-in-Reihe korrekt erkennen, Kandidatenzüge korrekt einschränken.
 - **TC-13–15 (MiniMax-KI):** Offensichtlichen Gewinnzug finden, drohenden Gewinn des Gegners blockieren.
- Erwähnen dass die Tests mit `'python -m pytest test_all.py'` ausgeführt werden.

Test-Infrastruktur

unittest · Temp-DB · Mocking

Damit Tests unabhängig voneinander und von der echten Datenbank laufen, verwenden wir Isolation durch temporäre Datenbanken:

DBTestCase

Jeder Testfall erbt von DBTestCase. In setUp() wird per tempfile.mkstemp() eine neue temporäre DB-Datei erstellt. Nach dem Test wird sie wieder gelöscht. So beeinflussen sich Tests nicht gegenseitig.

patch.object

Mit unittest.mock.patch.object wird database.DB_PATH auf den temporären Pfad umgeleitet. Das bedeutet: der getestete Code läuft unverändert, schreibt aber in die Temp-DB statt in die echte games.db.

pytest-Kompatibilität

Die Tests sind als unittest.TestCase geschrieben, können aber direkt mit pytest ausgeführt werden – pytest erkennt unittest-Tests automatisch.

■ Falls jemand fragt: Mocking verhindert dass Tests die Produkt-DB verändern oder voneinander abhängen.

Zusammenfassung

Was haben wir erreicht?

Zusammenfassend haben wir alle wesentlichen Projektziele erreicht:

- ■ **Zwei vollständige Strategiespiele** auf einem 6×6-Spielfeld – vollständig spielbar gegen eine KI.
 - ■ **Wiederverwendbarer MiniMax-Algorithmus** mit Alpha-Beta-Pruning – eine Engine für alle Spiele.
 - ■ **Sichere Authentifizierung** mit SHA-256-Hashing und zufälligem Salt – keine Klartextpasswörter.
 - ■ **SQLite-Datenbank** ohne externe Abhängigkeiten – die DB wird automatisch erstellt.
 - ■ ■ **tkinter-GUI** mit eigenem Design-System – läuft auf Windows, macOS und Linux.
 - ■ **15 Unittests** sichern die Kernfunktionen ab und können jederzeit ausgeführt werden.
- **Ruhig sprechen – das ist die Zusammenfassung, kein neues Material.**

SOLL/IST Feature-Vergleich

Lastenheft → Pflichtenheft → Umsetzung → Test

MUSS-Ziele: 12 von 12 ■

Alle Pflichtanforderungen wurden vollständig umgesetzt: beide Spiele, 5 Schwierigkeitsgrade, Login, Registrierung, Gastmodus, Spielabbruch mit Dialog, Regelanzeige, Bestenliste, SHA-256-Hashing, MiniMax-KI und Unittests.

SOLL-Ziele: 2 von 2 ■

Beide Soll-Anforderungen wurden erfüllt: Alpha-Beta-Pruning als MiniMax-Optimierung und die Bestenliste über SQL-Abfragen aus der Datenbank.

KANN-Ziele: 2 von 4 ■

- ■ **Sprachwechsel EN/DE** und Sprachpräferenz in der Datenbank sind implementiert.
- ■ **Dame als drittes Spiel** und eine alternative KI-Schnittstelle wurden aus Zeitgründen nicht umgesetzt.

Insgesamt: Alle MUSS- und SOLL-Anforderungen sind erfüllt. Die nicht umgesetzten KANN-Ziele waren von Anfang an als optional eingestuft.

■ **Nicht entschuldigen für die KANN-Ziele – sie waren optional und das ist legitim.**

SOLL/IST Projektplanung

Wasserfallmodell · 2 Meetings · Discord

Das Projekt wurde nach dem Wasserfallmodell geplant und in drei Phasen durchgeführt:

- **Analysephase (Woche 1: 20.04.–24.04.)** ■ plangemäß: Pflichtenheft, Work Breakdown Structure und Zeitplan wurden erstellt.
- **Implementierung (27.04.–04.05.)** ■ +1 Tag: Durch den Feiertag am 01. Mai wurde der 04. Mai als zusätzlicher Arbeitstag genutzt.
- **Test & Dokumentation** ■ parallel: Entgegen der ursprünglichen Planung liefen Test und Dokumentation parallel zur Implementierung – das war pragmatischer.
- **Abschluss (Woche 3)** ■ plangemäß: Abschlusspräsentation wie geplant.

Abweichungen

Statt eines Kanban-Boards wurde ein Discord-Channel als leichtgewichtige Alternative genutzt. Die zwei geplanten Team-Meetings fanden am 27.04. und 29.04. jeweils um 10:00 Uhr statt.

■ **Abweichungen positiv formulieren: 'wir haben uns angepasst' statt 'wir haben den Plan nicht eingehalten'.**

Benutzeroberfläche (GUI)

tkinter · HHBKTendo Design System · Dark Theme

Screen-Flow

- ① **Login-Screen:** Nutzer können sich einloggen, registrieren oder als Gast spielen.
- ② **Hauptmenü:** Spiel auswählen, Schwierigkeitsgrad einstellen, starten oder Bestenliste öffnen.
- ③ **Spielscreen:** 6×6-Spielfeld, Züge per Mausklick, KI-Zug wird nach kurzer Denkzeit angezeigt.

Design-System

- Deep Navy (#0D0D1A) als Hintergrund · Hot Pink (#E94560) für Primärelemente
- Royal Purple (#6C3FC5) als Sekundärfarbe · Cyber Cyan (#00D4FF) für Hover und Akzente

Technische Besonderheiten

- Buttons sind tk.Label mit Bindings implementiert – weil tk.Button auf macOS bg/fg-Farben ignoriert.
 - KI-Zug läuft in separatem Thread, GUI bleibt reaktionsfähig. after(0, callback) bringt das Ergebnis sicher in den Hauptthread zurück.
- Falls Demo: kurz das Dark Theme und die Farbgebung erwähnen.

Quellen & Werkzeuge

Entwicklungsumgebungen

- PyCharm (JetBrains) und Visual Studio Code (Microsoft)

KI-Assistenten

- Claude (Anthropic) und ChatGPT (OpenAI) wurden zur Unterstützung bei der Entwicklung genutzt.

Python-Bibliotheken

- Alle Bibliotheken außer Pillow sind Teil der Python-Standardbibliothek: tkinter, sqlite3, hashlib, threading, unittest, tempfile.
- Pillow wurde für GIF-Animationen in der GUI verwendet.

Quellen

- Wikipedia: Artikel zu MiniMax-Algorithmus und Alpha-Beta-Pruning
- HHBK Lernfeld 5 – Unterrichtsmaterial
- docs.python.org – offizielle Python-Dokumentation

Projektkoordination

- Zwei Team-Meetings (27.04. & 29.04., 10:00 Uhr), tägliche Kommunikation über Discord und WhatsApp.

Fazit – Projektverlauf

Team · Highlights · Lessons Learnt

Team & Rollenverteilung

- Lucas – Projektleitung, auth.py, Testing
- Jesse – database.py, Pitch, Testing · Arda – database.py, Testing
- Uilliam – GUI (main.py), Design · Jakub – pawn_chess.py
- Niklas – minimax.py · Jamie – tictactoe.py

Highlights

- Die spielunabhängige MiniMax-Schnittstelle war die eleganteste technische Lösung – ein KI-Code für alle Spiele.
- Die Teamarbeit verlief reibungslos dank eines klaren Pflichtenhefts und gegenseitiger Unterstützung.

Lessons Learnt

- Ein Kanban-Board früher einsetzen hätte die Aufgabenverteilung transparenter gemacht.
- Schnittstellen zwischen Modulen (z.B. Callback-Signaturen) detaillierter im Pflichtenheft definieren.

Ausblick

- Dame als drittes Spiel wäre mit dem bestehenden Callback-System einfach umsetzbar.
 - Sprach-Auswahl im Menü und weiterer Ausbau des Design-Systems.
- **Zum Abschluss: ruhig und selbstbewusst schließen. Das Projekt ist vollständig.**